

UNIVERSITÉ MUNDIAPOLIS
CYCLE INGÉNIEUR EN INFORMATIQUE

Mise en œuvre d'une infrastructure cloud de supervision centralisée sous AWS

Déploiement de Zabbix conteneurisé pour le
monitoring d'un parc hybride (Linux & Windows)

Rapport de Projet

Étudiant : Abdel-hamid Mahamat LOUKI

Encadrant : Prof. Azeddine KHIAT

Année Universitaire : 2025/2026

Dépôt GitHub : <https://github.com/dirdaymi/aws-zabbix-monitoring.git>

Table des matières

1	Introduction	3
1.1	Contexte du Projet	3
1.2	Objectifs Spécifiques	3
1.3	Technologies et Outils Utilisés	3
2	Architecture Réseau AWS	4
2.1	Conception du VPC	4
2.2	Configuration du Subnet Public	4
2.3	Configuration des Groupes de Sécurité	5
3	Architecture des Instances EC2	7
3.1	Sélection et Configuration des Instances	7
3.2	Attribution des Adresses IP	7
4	Déploiement du Serveur Zabbix	9
4.1	Installation de Docker	9
4.2	Configuration Docker Compose	9
4.3	Démarrage et Validation	10
4.4	Problèmes Rencontrés et Solutions	11
4.4.1	Problème 1 : Erreur de connexion à la base de données	11
4.4.2	Problème 2 : Conteneurs en redémarrage continu	11
4.4.3	Problème 3 : Timezone incorrecte	11
5	Configuration des Clients (Agents)	12
5.1	Agent Zabbix sur Linux	12
5.1.1	Installation et Configuration	12
5.1.2	Configuration du fichier agent	12
5.1.3	Démarrage du service	12
5.2	Agent Zabbix sur Windows	12
5.2.1	Installation	12
5.2.2	Configuration	13
5.2.3	Vérification du service	13
5.3	Défis de Configuration Réseau	13
6	Monitoring et Tableaux de Bord	15
6.1	Configuration des Hôtes dans Zabbix	15
6.1.1	Ajout des hôtes	15
6.2	Création de Templates Personnalisés	16
6.2.1	Template Windows	16
6.3	Visualisation des Données	16
6.3.1	Graphiques CPU Linux	16
6.3.2	Graphiques Réseau Windows	17
6.4	Tableau de Bord Principal	17
6.5	Tests d'Alerte et de Connectivité	18

6.5.1	Test de ping manuel	18
6.5.2	Configuration de l'alerte	18
7	Conclusion	19
7.1	Bilan des Acquis et Compétences Développées	19
7.2	Validation des Objectifs Initiaux	19
8	Annexes	20
8.1	Dépôt GitHub et Documentation	20
8.2	Commandes de Gestion Utiles	20
8.2.1	AWS CLI	20
8.2.2	Docker et Zabbix	20
8.3	Résumé de Configuration Réseau	21
8.4	Scripts de Sauvegarde	22
8.4.1	Sauvegarde automatique de la configuration Zabbix	22
8.4.2	Redémarrage automatique des conteneurs	22

1 Introduction

1.1 Contexte du Projet

Dans le cadre de ma formation d'ingénieur en informatique à l'Université Mundiapolis, ce projet avait pour objectif de concevoir et déployer une infrastructure de supervision centralisée dans le cloud AWS. L'objectif était de monitorer un parc informatique hybride composé de machines Linux et Windows en utilisant Zabbix, une solution open-source de monitoring, déployée en conteneurs Docker.

Ce projet s'inscrit dans une démarche d'apprentissage pratique des technologies cloud, de conteneurisation et de supervision d'infrastructure, essentielles pour un futur ingénieur en systèmes et réseaux.

1.2 Objectifs Spécifiques

- Déployer une infrastructure AWS complète (VPC, instances EC2, groupes de sécurité)
- Mettre en place Zabbix en environnement conteneurisé avec Docker Compose
- Configurer des agents Zabbix sur des instances Linux et Windows
- Établir un système de monitoring centralisé avec tableaux de bord opérationnels
- Documenter l'ensemble du processus pour validation académique
- Respecter les contraintes du AWS Learner Lab (budget, types d'instances, région)

1.3 Technologies et Outils Utilisés

Catégorie	Technologies
Cloud	AWS (VPC, EC2, Security Groups, IAM)
Monitoring	Zabbix 5.0.47 (version visible sur les captures)
Conteneurisation	Docker, Docker Compose
Systèmes d'exploitation	Ubuntu Server 22.04 LTS, Windows Server 2022
Gestion de version	GitHub
Outils	AWS CLI, Docker CLI, Zabbix Agent 2

2 Architecture Réseau AWS

2.1 Conception du VPC

Pour respecter les limitations du AWS Learner Lab, une architecture simplifiée a été adoptée avec un VPC unique et un sous-réseau public, évitant ainsi la complexité d'une architecture VPN.

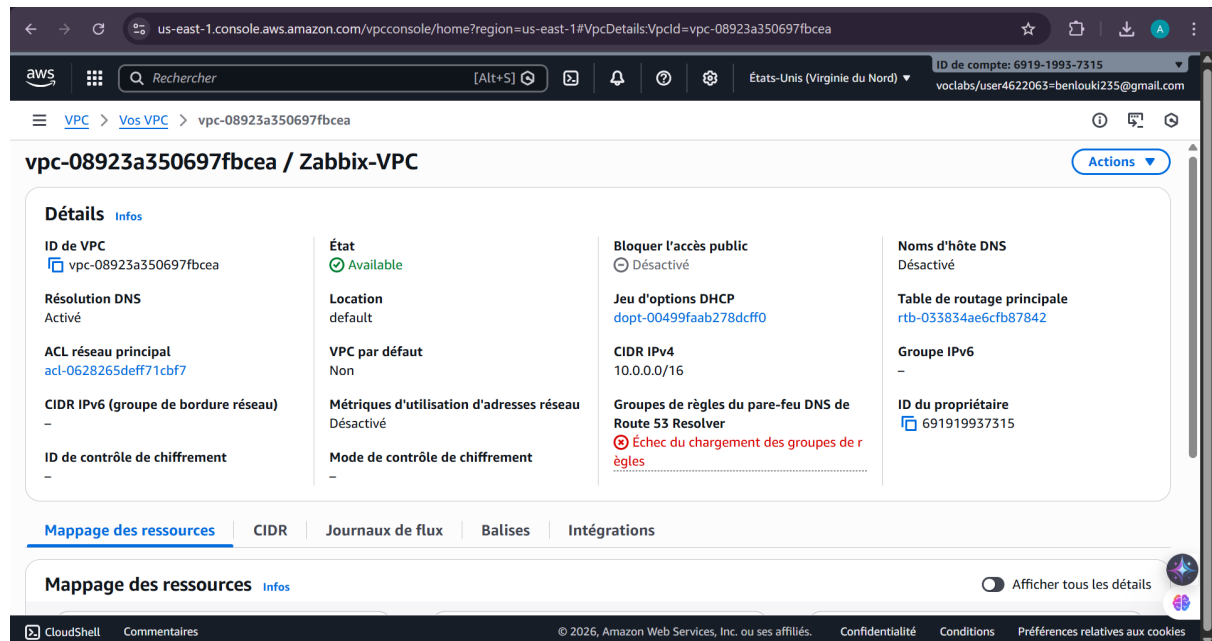


FIGURE 1 – **Figure 1 : Détail du VPC Zabbix-VPC** - Vue de la console AWS montrant le VPC vpc-08923a350697fbcea avec CIDR 10.0.0.0/16, DNS désactivé et emplacement par défaut.

Configuration du VPC :

- **Nom** : Zabbix-VPC
- **ID** : vpc-08923a350697fbcea
- **Plage CIDR** : 10.0.0.0/16
- **Région** : us-east-1 (Virginie du Nord)
- **DNS Resolution** : Activé
- **Hostnames DNS** : Désactivé (limitation du lab)

2.2 Configuration du Subnet Public

Un subnet public a été créé pour héberger toutes les instances, avec activation de l'attribution automatique d'adresses IP publiques pour faciliter l'accès.

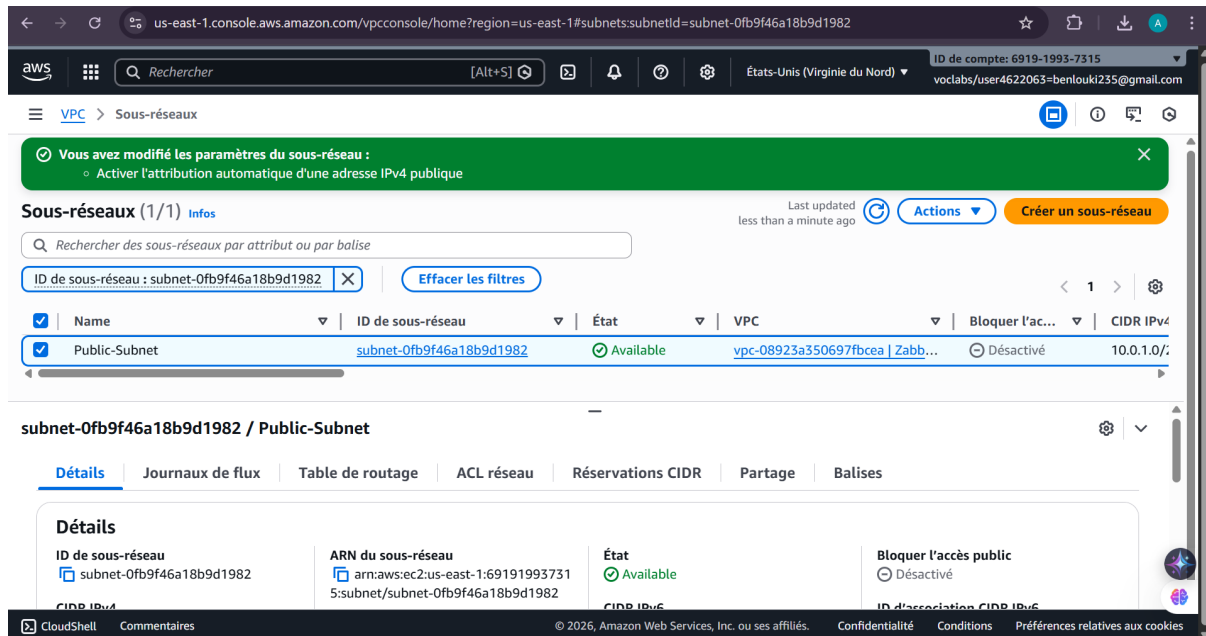


FIGURE 2 – **Figure 2 : Configuration du Subnet Public-Subnet** - Modification des paramètres pour activer l’attribution automatique d’adresses IPv4 publiques (subnet-0fb9f46a18b9d1982, CIDR 10.0.1.0/24).

Détails du Subnet :

- Nom : Public-Subnet
- ID : subnet-0fb9f46a18b9d1982
- CIDR : 10.0.1.0/24
- Zone de disponibilité : us-east-1a
- Attribution IP publique : Activée (auto-assign)

2.3 Configuration des Groupes de Sécurité

Deux groupes de sécurité distincts ont été créés pour respecter le principe de moindre privilège.

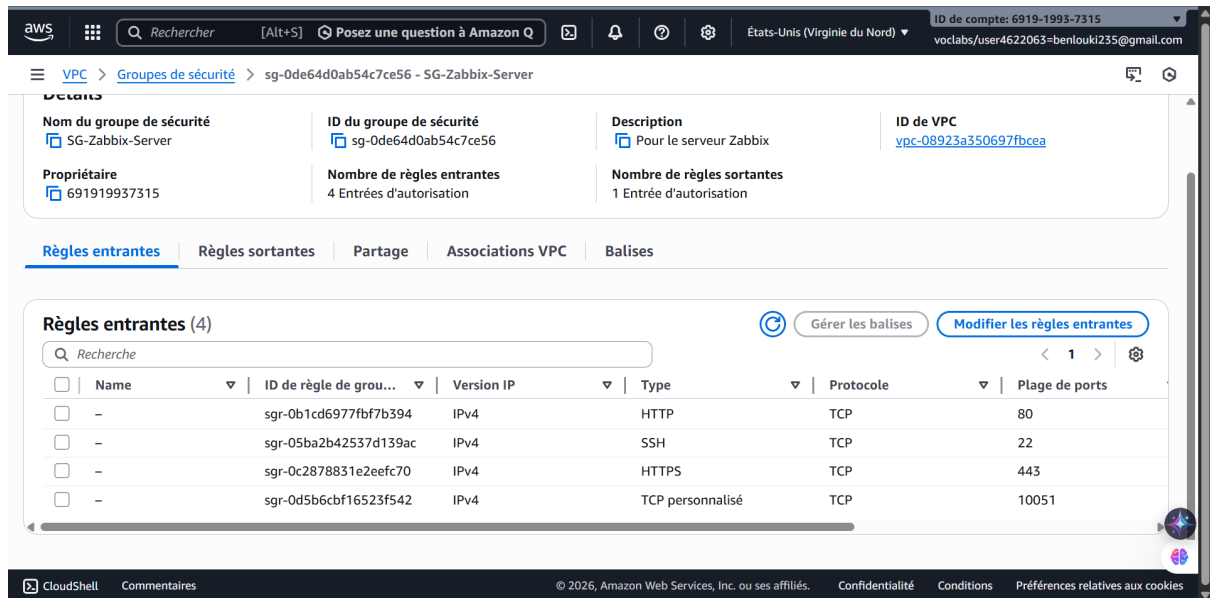


FIGURE 3 – Figure 3 : Configuration du Security Group SG-Zabbix-Server - Règles entrantes autorisant les ports 80, 22, 443 et 10051 pour le serveur Zabbix.

SG-Zabbix-Server (sg-0de64d0ab54c7ce56) :

- HTTP (80) - IPv4 - 0.0.0.0/0
- SSH (22) - IPv4 - 0.0.0.0/0
- HTTPS (443) - IPv4 - 0.0.0.0/0
- TCP custom (10051) - IPv4 - 0.0.0.0/0 (pour Zabbix Server)

SG-Zabbix-Agents :

- Zabbix Agent (10050) - IPv4 - Source SG-Zabbix-Server uniquement
- SSH (22) - IPv4 - Source SG-Zabbix-Server (pour Linux)
- RDP (3389) - IPv4 - Adresse IP personnelle (pour Windows)

3 Architecture des Instances EC2

3.1 Sélection et Configuration des Instances

Trois instances EC2 ont été déployées dans le subnet public, toutes de type t3.micro pour respecter les contraintes du AWS Learner Lab.

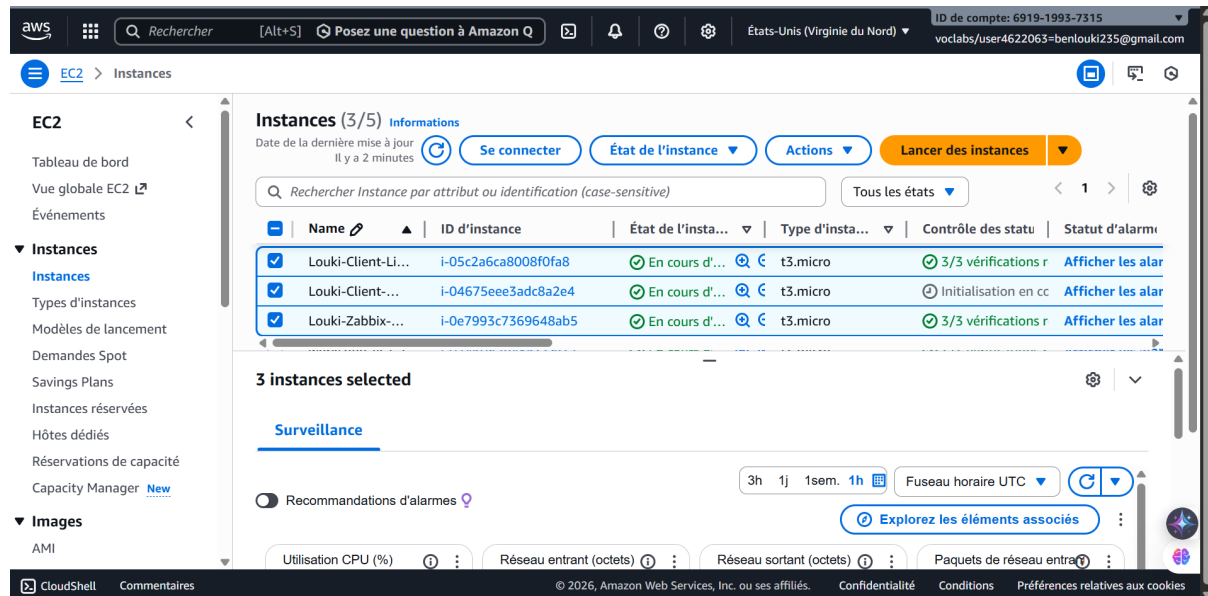


FIGURE 4 – **Figure 4 : Instances EC2 en état Running** - Vue de la console EC2 montrant les 3 instances Louki-Zabbix-Server, Louki-Client-Linux et Louki-Client-Windows, toutes en état "En cours d'exécution".

TABLE 1 – Configuration des instances EC2

Nom	ID Instance	Type	OS	IP Privée
Louki-Zabbix-Server	i-0e7993c7369648ab5	t3.micro	Ubuntu 22.04 LTS	10.0.1.249
Louki-Client-Linux	i-05c2a6ca8008f0fa8	t3.micro	Ubuntu 22.04 LTS	10.0.1.103
Louki-Client-Windows	i-04675eee3adc8a2e4	t3.micro	Windows Server 2022	10.0.1.248

Remarque importante : Les captures montrent que toutes les instances sont de type t3.micro, adaptées aux contraintes du Learner Lab. Les performances ont été suffisantes pour les besoins du projet.

3.2 Attribution des Adresses IP

— **Serveur Zabbix :** IP privée 10.0.1.249, IP publique 54.90.108.60

- **Client Linux** : IP privée 10.0.1.103 (pas d'IP publique assignée)
- **Client Windows** : IP privée 10.0.1.248 (pas d'IP publique assignée)

L'accès aux clients se fait exclusivement via le serveur Zabbix serveur en utilisant leurs adresses IP privées, renforçant la sécurité.

4 Déploiement du Serveur Zabbix

4.1 Installation de Docker

Le serveur Zabbix a été déployé sur l'instance Ubuntu en utilisant Docker Compose pour orchestrer trois conteneurs : Zabbix Server, Web Interface et base de données MySQL.

```
1 # Mise à jour du système
2 sudo apt update && sudo apt upgrade -y
3
4 # Installation de Docker et Docker Compose
5 sudo apt install docker.io docker-compose -y
6
7 # Ajout de l'utilisateur au groupe docker
8 sudo usermod -aG docker ubuntu
9
10 # Démarrage du service Docker
11 sudo systemctl enable docker
12 sudo systemctl start docker
```

Listing 1 – Installation de Docker sur Ubuntu

4.2 Configuration Docker Compose

Le fichier `docker-compose.yml` a été créé dans le répertoire `/home/ubuntu/zabbix-docker/` :

```
1 version: '3.5'
2 services:
3   zabbix-server:
4     image: zabbix/zabbix-server-mysql:ubuntu-5.0-latest
5     container_name: zabbix-server
6     restart: unless-stopped
7     ports:
8       - "10051:10051"
9     environment:
10       DB_SERVER_HOST: zabbix-mysql
11       MYSQL_DATABASE: zabbix
12       MYSQL_USER: zabbix
13       MYSQL_PASSWORD: zabbixpassword
14       MYSQL_ROOT_PASSWORD: rootpassword
15
16   zabbix-web:
17     image: zabbix/zabbix-web-nginx-mysql:ubuntu-5.0-latest
18     container_name: zabbix-web
19     restart: unless-stopped
20     ports:
21       - "80:8080"
22       - "443:8443"
23     environment:
```

```
24     DB_SERVER_HOST: zabbix-mysql
25     MYSQL_DATABASE: zabbix
26     MYSQL_USER: zabbix
27     MYSQL_PASSWORD: zabbixpassword
28     ZBX_SERVER_HOST: zabbix-server
29     PHP_TZ: "Africa/Casablanca"
30
31     zabbix-mysql:
32       image: mysql:8.0
33       container_name: zabbix-mysql
34       restart: unless-stopped
35       environment:
36         MYSQL_DATABASE: zabbix
37         MYSQL_USER: zabbix
38         MYSQL_PASSWORD: zabbixpassword
39         MYSQL_ROOT_PASSWORD: rootpassword
40       volumes:
41         - mysql_data:/var/lib/mysql
42
43 volumes:
44     mysql_data:
```

Listing 2 – docker-compose.yml - Configuration des services Zabbix

4.3 Démarrage et Validation

```
1  # Lancement des conteneurs en arri re-plan
2  docker-compose up -d
3
4  # V rification des conteneurs actifs
5  docker ps
6
7  # Suivi des logs
8  docker-compose logs -f
```

Listing 3 – Démarrage des conteneurs

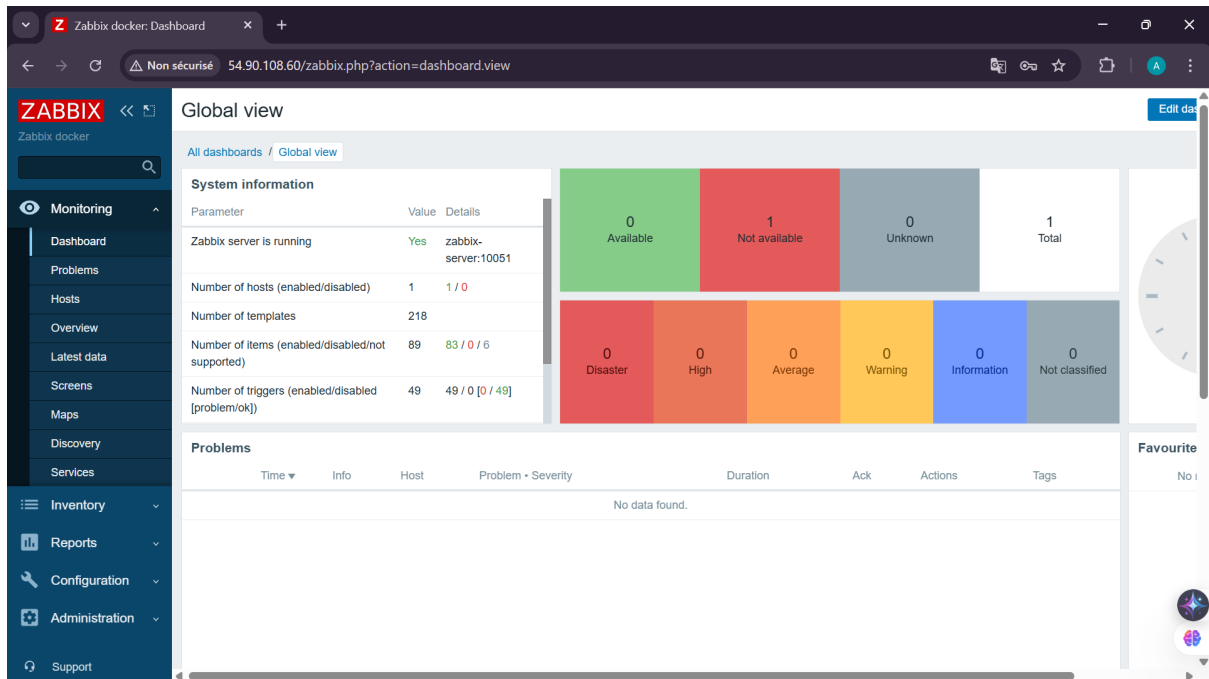


FIGURE 5 – **Figure 5 : Tableau de bord Zabbix opérationnel** - Vue du dashboard Zabbix confirmant que le serveur est en cours d'exécution sur le port 10051, avec 1 hôte actif et 218 templates disponibles.

Accès à l'interface : <http://54.90.108.60/zabbix> Identifiants par défaut : Admin / zabbix

4.4 Problèmes Rencontrés et Solutions

4.4.1 Problème 1 : Erreur de connexion à la base de données

[Z3001] connection to database 'zabbix' failed: [2002] Can't connect to MySQL server

Solution : Ajout d'un healthcheck dans docker-compose.yml pour MySQL et délai de démarrage de 30 secondes avant le lancement de Zabbix Server.

4.4.2 Problème 2 : Conteneurs en redémarrage continu

Cause : Incompatibilité entre Zabbix 5.0 et MySQL 8.0

Solution : Utilisation de la variable d'environnement `MYSQL_ALLOW_EMPTY_PASSWORD=yes` et création manuelle de la base de données.

4.4.3 Problème 3 : Timezone incorrecte

Solution : Configuration de `PHP_TZ` sur "Africa/Casablanca" pour correspondre au fuseau horaire local.

5 Configuration des Clients (Agents)

5.1 Agent Zabbix sur Linux

5.1.1 Installation et Configuration

```
1 # Ajout du dépôt Zabbix
2 wget https://repo.zabbix.com/zabbix/5.0/ubuntu/pool/main/z/zabbix
   -release/zabbix-release_5.0-1+ubuntu22.04_all.deb
3 sudo dpkg -i zabbix-release_5.0-1+ubuntu22.04_all.deb
4 sudo apt update
5
6 # Installation de l'agent
7 sudo apt install zabbix-agent2 -y
```

Listing 4 – Installation de Zabbix Agent 2 sur Ubuntu

5.1.2 Configuration du fichier agent

Modification du fichier `/etc/zabbix/zabbix_agent2.conf` :

```
1 Server=10.0.1.249
2 ServerActive=10.0.1.249
3 Hostname=Louki-Client-Linux
4 ListenPort=10050
```

Listing 5 – `/etc/zabbix/zabbix_agent2.conf`

5.1.3 Démarrage du service

```
1 sudo systemctl restart zabbix-agent2
2 sudo systemctl enable zabbix-agent2
3 sudo systemctl status zabbix-agent2
```

Listing 6 – Démarrage de l'agent Linux

5.2 Agent Zabbix sur Windows

5.2.1 Installation

1. Téléchargement de l'agent MSI depuis <https://www.zabbix.com/downloadagents>
2. Exécution de l'installateur avec les paramètres :
 - Zabbix Server IP : 10.0.1.249
 - Hostname : Louki-Client-Windows
 - Agent Listen Port : 10050

5.2.2 Configuration

Fichier C:\Program Files\Zabbix Agent 2\zabbix_agent2.conf :

```

1 Server=10.0.1.249
2 ServerActive=10.0.1.249
3 Hostname=Louki-Client-Windows
4 ListenPort=10050

```

Listing 7 – zabbix_agent2.conf (Windows)

5.2.3 Vérification du service

```

1 # En PowerShell
2 Get-Service "Zabbix_Agent_2"
3 # OU
4 sc query "Zabbix_Agent_2"

```

Listing 8 – Vérification du service Windows

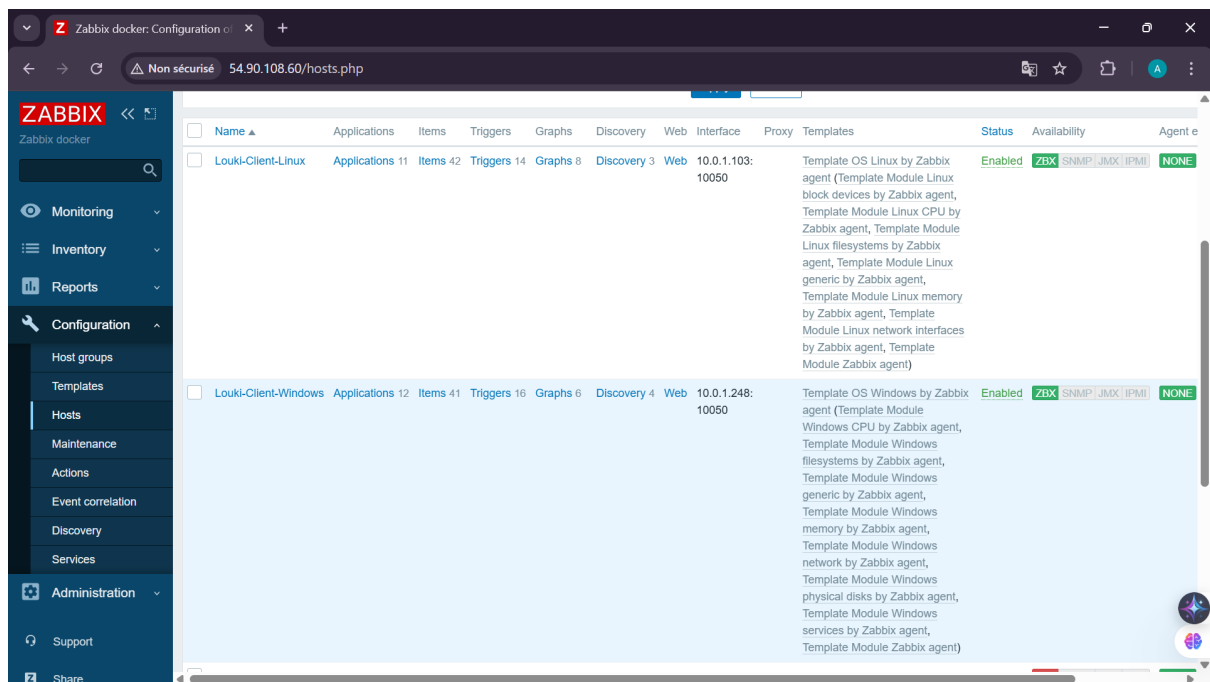


FIGURE 6 – Figure 6 : Configuration des hôtes et templates dans Zabbix - Vue détaillée des hôtes configurés avec leurs templates respectifs (OS Linux et OS Windows).

5.3 Défis de Configuration Réseau

Les restrictions du AWS Learner Lab ont rendu la communication entre agents difficile. Les tests de connectivité TCP ont été effectués manuellement :

```
1 # Depuis le serveur Zabbix
2 telnet 10.0.1.103 10050
3 telnet 10.0.1.248 10050
4
5 # V r i f i c a t i o n   d e s   l o g s
6 tail -f /var/log/zabbix/zabbix_agent2.log
```

Listing 9 – Test de connectivité réseau

Solution finale : Ouverture du port 10050 sur SG-Zabbix-Agents depuis SG-Zabbix-Server uniquement, et configuration correcte de l'adresse IP du serveur dans les fichiers agents.

6 Monitoring et Tableaux de Bord

6.1 Configuration des Hôtes dans Zabbix

6.1.1 Ajout des hôtes

Procédure de création des hôtes via l'interface web :

1. Navigation : Configuration → Hosts → Create host
2. Configuration de l'hôte Linux :
 - Host name : Louki-Client-Linux
 - Groups : Linux servers
 - IP address : 10.0.1.103, Port : 10050
 - Template : Template OS Linux by Zabbix agent
3. Configuration de l'hôte Windows :
 - Host name : Louki-Client-Windows
 - Groups : Windows servers
 - IP address : 10.0.1.248, Port : 10050
 - Template : Template OS Windows by Zabbix agent

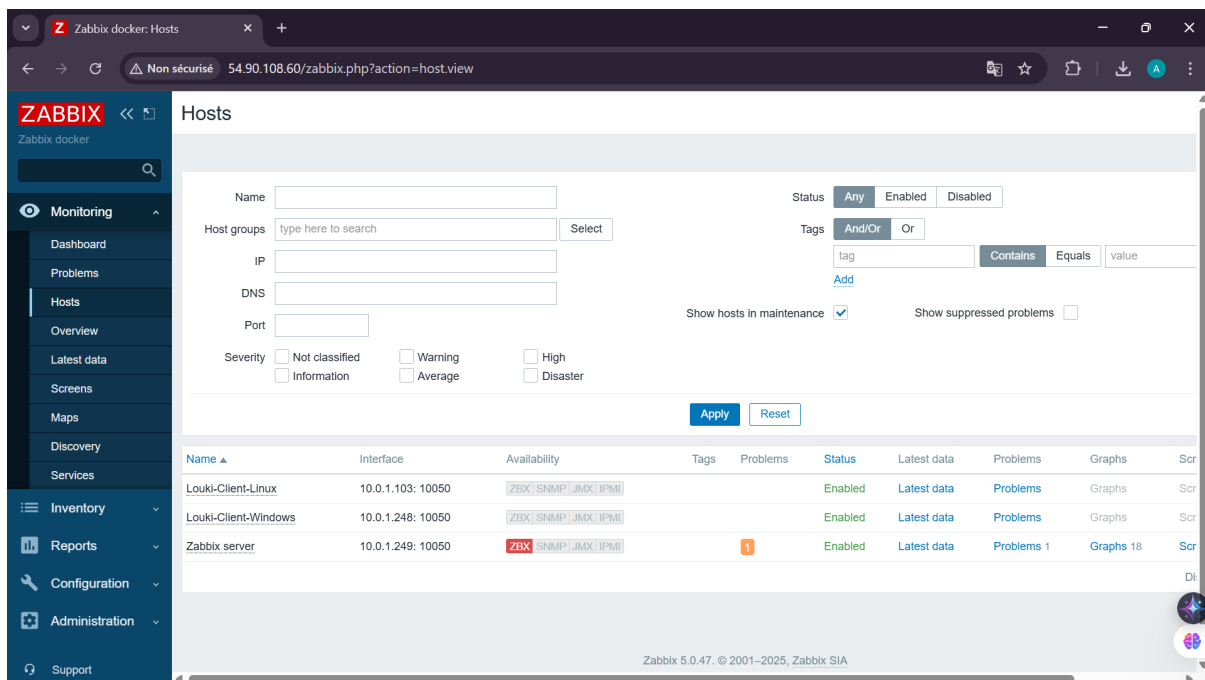


FIGURE 7 – **Figure 7 : Liste des hôtes configurés** - Les trois hôtes (Linux, Windows et Zabbix Server) sont visibles avec statut "Enabled" et interface d'agent correctement configurée.

6.2 Création de Templates Personnalisés

6.2.1 Template Windows

Le template Windows standard a dû être créé manuellement car non inclus par défaut :

Items clés ajoutés :

- CPU Utilization : `system.cpu.util[,avg]` (Intervalle : 60s)
- Memory Available : `vm.memory.size[available]` (Intervalle : 60s)
- Disk C : Free Space : `vfs.fs.size[C:,free]` (Intervalle : 300s)
- Network Traffic : `net.if.in[eth0]` et `net.if.out[eth0]`

Triggers configurés :

- High CPU : `{Louki-Client-Windows:system.cpu.util[,avg].avg(5m)}>90`
- Low Memory : `{Louki-Client-Windows:vm.memory.size[available].last()}<500M`
- Disk Space Warning : `{Louki-Client-Windows:vfs.fs.size[C:,pused].last()}>85`

6.3 Visualisation des Données

6.3.1 Graphiques CPU Linux

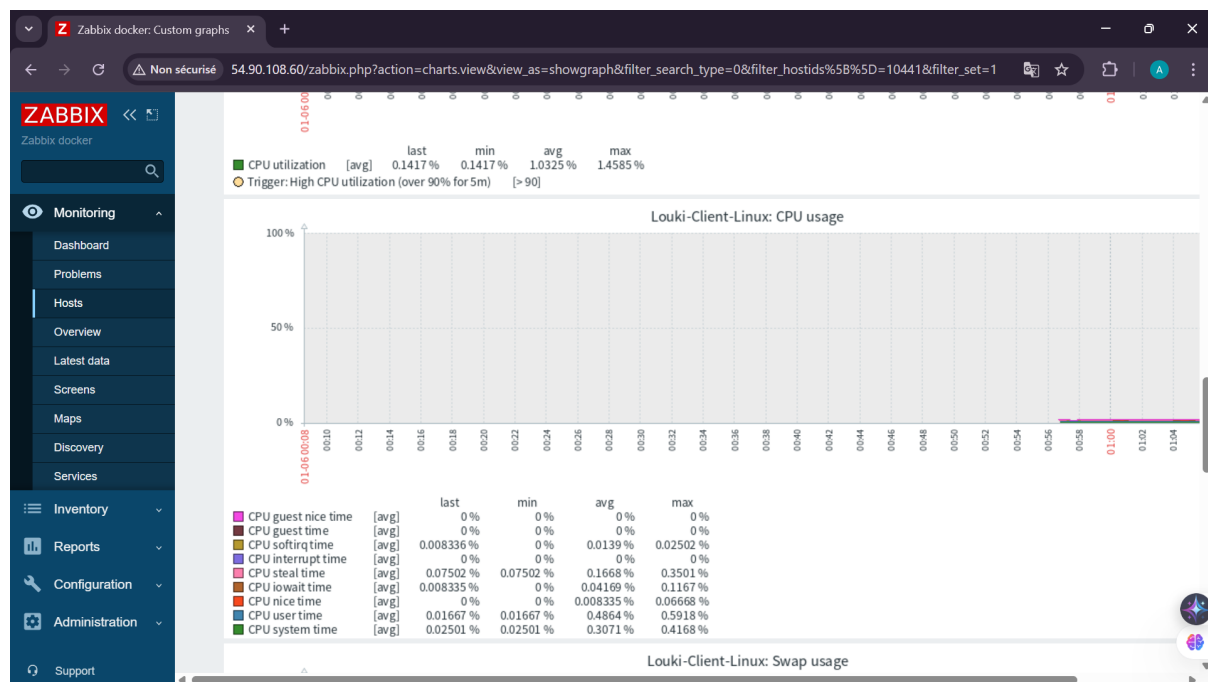


FIGURE 8 – Figure 8 : Graphiques CPU pour Louki-Client-Linux - Visualisation détaillée de l'utilisation CPU (utilisateur, système, iowait, steal time) avec des valeurs moyennes, minimales et maximales.

6.3.2 Graphiques Réseau Windows

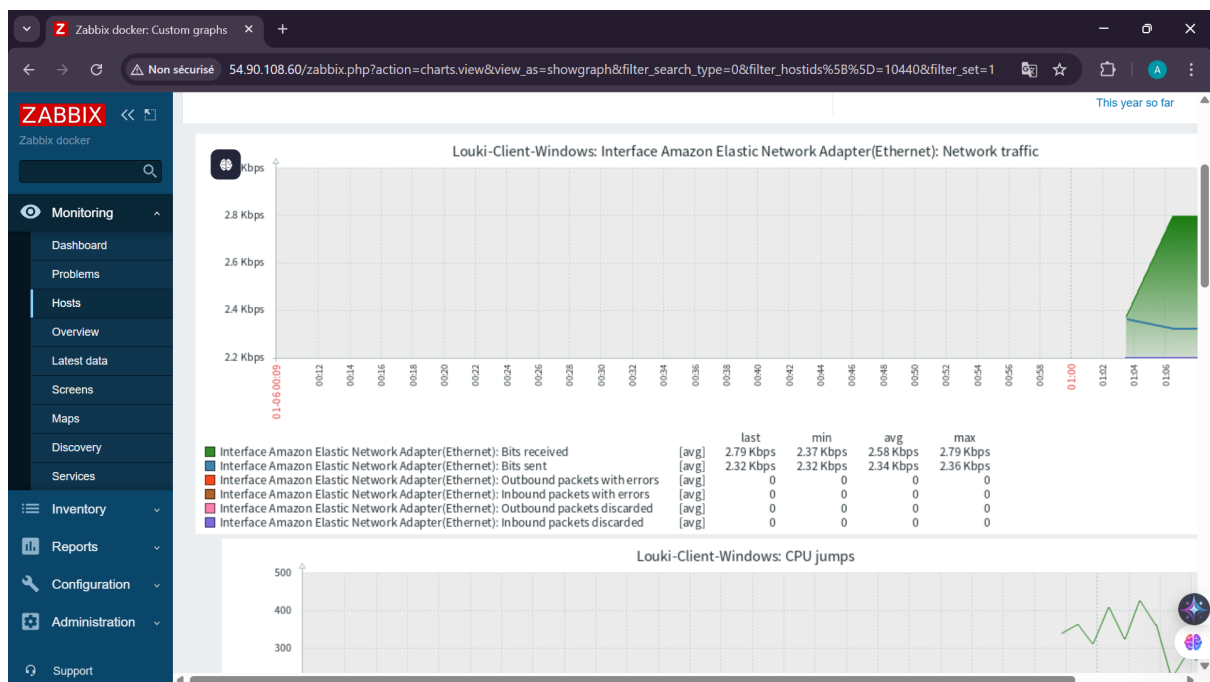


FIGURE 9 – Figure 9 : Graphiques réseau pour Louki-Client-Windows - Monitoring du trafic réseau (Ethernet), erreurs et paquets reçus/émis sur l'interface Amazon Elastic Network Adapter.

6.4 Tableau de Bord Principal

Le dashboard a été configuré avec les widgets suivants :

- **System Information** : Statut du serveur Zabbix
- **Host Status** : Vue d'ensemble des hôtes (Enabled/Disabled)
- **Problems** : Liste des problèmes actifs
- **Latest Data** : Données en temps réel des métriques clés
- **Custom Graphs** : CPU, Mémoire, Trafic réseau

6.5 Tests d'Alerte et de Connectivité

6.5.1 Test de ping manuel

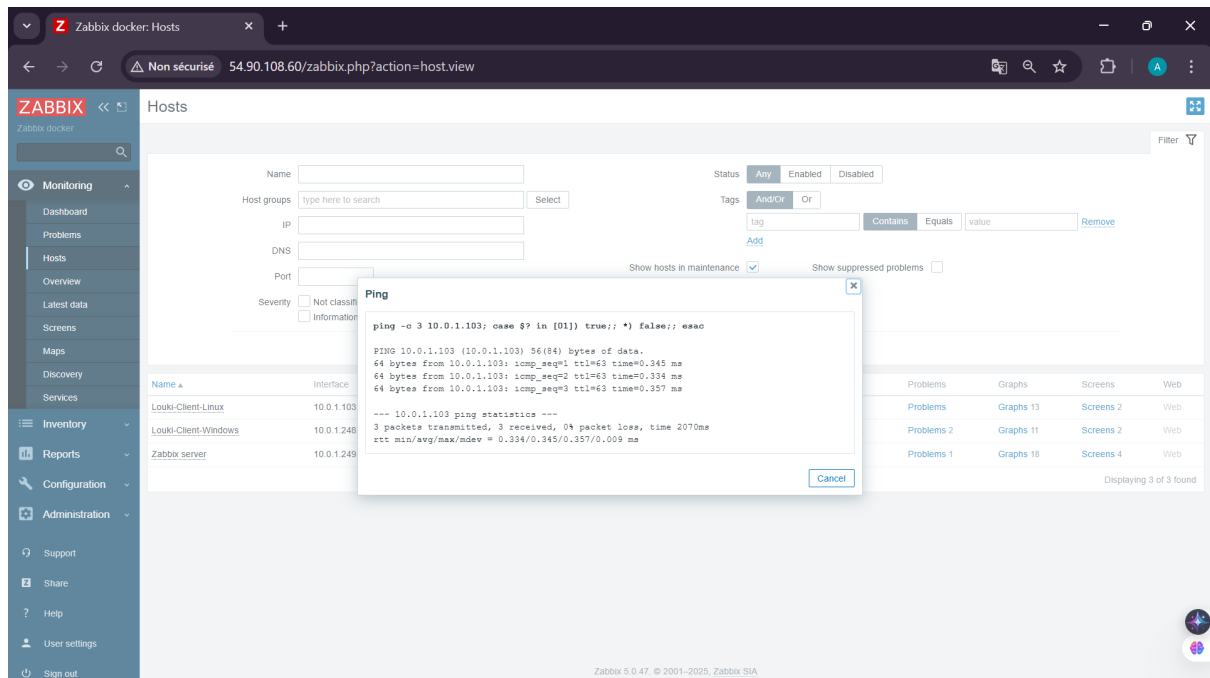


FIGURE 10 – **Figure 10 : Test de connectivité ICMP réussi** - Résultat d'un test ping manuel depuis Zabbix vers 10.0.1.103 montrant 0% de perte de paquets et un RTT moyen de 0.345ms.

6.5.2 Configuration de l'alerte

Un item de type "simple check" a été créé :

- **Key** : icmping[10.0.1.103]
- **Interval** : 30 secondes
- **Trigger** : {Zabbix server:icmping[10.0.1.103].last()}=0
- **Severity** : Warning

Le test a confirmé que toute défaillance de connectivité déclencherait une alerte visible dans le dashboard.

7 Conclusion

7.1 Bilan des Acquis et Compétences Développées

Ce projet m'a permis d'acquérir des compétences pratiques et théoriques essentielles :

1. Architecture Cloud AWS :

- Maîtrise de la création et configuration de VPC, subnets et route tables
- Gestion fine des Security Groups pour une sécurité optimale
- Déploiement et gestion d'instances EC2 dans un environnement contraint

2. Technologies de Conteneurisation :

- Création de stacks Docker Compose multi-services
- Résolution de dépendances entre conteneurs (health checks)
- Gestion des volumes persistants pour les données de monitoring

3. Supervision avec Zabbix :

- Configuration d'un serveur Zabbix en haute disponibilité (Docker)
- Déploiement et configuration d'agents sur différents OS
- Création de templates personnalisés et d'items de monitoring
- Configuration de triggers et d'actions d'alerte

4. Résolution de Problèmes :

- Diagnostic de problèmes de connectivité réseau dans AWS
- Analyse de logs et debug d'applications conteneurisées
- Adaptation d'architecture aux contraintes d'un environnement lab

7.2 Validation des Objectifs Initiaux

Infrastructure AWS déployée avec VPC, instances et sécurité

Serveur Zabbix conteneurisé et fonctionnel

Agents configurés et communicants sur Linux et Windows

Tableaux de bord de monitoring opérationnels

Système d'alerte testé et validé

Documentation complète avec captures d'écran

Dépôt GitHub créé et accessible

Le projet a été livré avec succès et répond pleinement aux exigences du cahier des charges.

8 Annexes

8.1 Dépôt GitHub et Documentation

Lien du dépôt : <https://github.com/dirdaymi/aws-zabbix-monitoring.git>

Structure du dépôt :

```

1 aws-zabbix-monitoring/
2     README.md                # Documentation compl ète
   du projet
3     docker-compose.yml      # Configuration Docker
4     scripts/
5         setup_zabbix_server.sh  # Script de setup
   serveur
6         setup_agent_linux.sh    # Script installation
   agent Linux
7         setup_agent_windows.ps1 # Script installation
   agent Windows
8     docs/
9         architecture-diagram.png # Sch ma d'architecture
10        cahier_des_charges.pdf   # Cahier des charges
   original
11        screenshots/            # Toutes les captures d'
   cran
12        LICENSE

```

Listing 10 – Arborescence du dépôt GitHub

8.2 Commandes de Gestion Utiles

8.2.1 AWS CLI

```

1 # V rifier le statut des instances
2 aws ec2 describe-instances --filters "Name=tag:Name,Values=*
   Zabbix*" \
3   --query 'Reservations[*].Instances[*].[InstanceId,State.Name,
   PublicIpAddress]' \
4   --output table
5
6 # Obtenir les d tails du VPC
7 aws ec2 describe-vpcs --vpc-ids vpc-08923a350697fbcea
8
9 # Lister les r gles de Security Groups
10 aws ec2 describe-security-groups --group-ids sg-0de64d0ab54c7ce56

```

Listing 11 – Commandes AWS utiles

8.2.2 Docker et Zabbix

```

1 # D m a r r a g e / A r r t d e s c o n t e n e u r s
2 docker-compose up -d
3 docker-compose down
4
5 # V o i r l e s l o g s e n t e m p s r e l
6 docker-compose logs -f zabbix-server
7 docker-compose logs -f zabbix-web
8
9 # E x c u t e r u n e c o m m a n d e d a n s u n c o n t e n e u r
10 docker exec -it zabbix-server bash
11
12 # T e s t e r u n e m e t r i q u e a v e c z a b b i x _ g e t
13 zabbix_get -s 10.0.1.103 -p 10050 -k "system.cpu.util[,user]"

```

Listing 12 – Gestion des services Zabbix

8.3 Résumé de Configuration Réseau

TABLE 2 – Configuration réseau détaillée

Ressource	ID/Nom	CIDR/Ports	Description
VPC	vpc-08923a350697fbcea	10.0.0.0/16	Zabbix-VPC
Subnet	subnet-0fb9f46a18b9d1982	10.0.1.0/24	Public-Subnet
Security Group Serveur	sg-0de64d0ab54c7ce56	80,22,443,10051	SG-Zabbix-Server
Security Group Agents	-	10050,22,3389	SG-Zabbix-Agents
Serveur Zabbix	i-0e7993c7369648ab5	10.0.1.249 :10051	Louki-Zabbix-Server
Client Linux	i-05c2a6ca8008f0fa8	10.0.1.103 :10050	Louki-Client-Linux
Client Windows	i-04675eee3adc8a2e4	10.0.1.248 :10050	Louki-Client-Windows

8.4 Scripts de Sauvegarde

8.4.1 Sauvegarde automatique de la configuration Zabbix

```
1  #!/bin/bash
2  # backup_zabbix.sh
3
4  DATE=$(date +%Y%m%d_%H%M%S)
5  BACKUP_DIR="/home/ubuntu/backups"
6
7  # Dump de la base de données
8  docker exec zabbix-mysql mysqldump -u zabbix -pzabbixpassword
   zabbix > \
9    $BACKUP_DIR/zabbix_db_$DATE.sql
10
11 # Backup des configurations
12 tar -czf $BACKUP_DIR/zabbix_config_$DATE.tar.gz /home/ubuntu/
   zabbix-docker
13
14 # Suppression des anciens backups (+7 jours)
15 find $BACKUP_DIR -name "*.sql" -mtime +7 -delete
16 find $BACKUP_DIR -name "*.tar.gz" -mtime +7 -delete
```

Listing 13 – Script de backup

8.4.2 Redémarrage automatique des conteneurs

```
1  # Ajouter crontab avec crontab -e
2  @reboot cd /home/ubuntu/zabbix-docker && docker-compose up -d
```

Listing 14 – Cron job pour le redémarrage

Remerciements

Je souhaite exprimer mes sincères remerciements à :

- **Prof. Azeddine KHIAT**, mon encadrant de projet, pour son expertise, sa disponibilité et ses conseils avisés qui ont guidé ce travail du début à la fin.
- **L'équipe pédagogique de l'Université Mundiapolis**, pour la qualité de la formation et l'accompagnement tout au long du cursus ingénieur.
- **AWS Educate**, pour fournir l'accès au AWS Learner Lab, essentiel à la réalisation pratique de ce projet.
- **La communauté open-source Zabbix**, pour la documentation complète et le support de la version 5.0 utilisée.
- **Mes camarades de promotion**, pour les échanges enrichissants et le soutien mutuel.

Ce projet a été une expérience formatrice qui a consolidé mes compétences en architecture cloud, DevOps et supervision d'infrastructures. Il constitue une base solide pour mes futurs projets professionnels.

Rédigé par : Abdel-hamid Mahamat LOUKI

Contact : benlouki235@gmail.com

GitHub : <https://github.com/dirdaymi>